

ORDRES BÀSIQUES PER UTILITZAR EL SCV GIT:

Utilitzarem un projecte com a repositori local. També treballarem amb repositoris globals. El que apareix en vermell són paràmetres que haurem de substituir pels valors que necessitem.

1. Configuració del repositori local

En primer lloc, cal fer que GIT consideri el nostre projecte com a repositori. Per això cal:

1. situar-se a la carpeta arrel del projecte (o la carpeta on estigui la nostra feina)
2. teclejar l'ordre:

```
git init
```

2. Configuració del repositori remot propi:

Per poder treballar amb repositoris remots cal indicar l'email, el nom d'usuari (la contrassenya la demanen en fer ...) i la URL del repositori amb:

```
git config --global user.name "nomUsuari"  
git config --global user.email "correuElectrònic"  
git remote add origin https://URLdelRepositori(amb extensió .git)
```

3. Excloure fitxers.

Possiblement voldrem excloure algun fitxer del sistema de control de versions. Per exemple, els fitxers .class. Això es fa posant aquestes extensions al fitxer .gitignore. Pot fer-se, per exemple, així:

```
echo *.class > .gitignore
```

4. Incloure fitxers al control de versions.

Cal utilitzar l'ordre

```
git add nomFitxerOcarpeta-admetCaractersComodí
```

Si volem afegir tot el que hi ha a la carpeta actual (incloent-hi les subcarpetes), podem fer:

```
git add .
```

Els fitxers inclosos es tindran en compte al proper *commit* o confirmació dels canvis realitzats).

5. Confirmar canvis

Si volem confirmar tots els canvis als fitxers inclosos amb *git add* des del darrer *commit*:

```
git commit -m "missatge de confirmació"
```

Si volem confirmar tots els fitxers modificats o esborrats des del darrer commit i que ja hi estaven inclosos, podem fer-ho així:

```
git commit -a -m "missatge de confirmació"
```

(si hem creat un fitxer nou, en no haver-lo afegit amb *git add* cap vegada, no s'hi inclouria).

6. Veure el que hi ha pendent de confirmar (entre altres informacions):

```
git status
```

7. Sincronitzar un repositori remot amb el nostre com a origen:

```
git push -u origin branca
```

branca és la branca que pugem. La branca principal, per defecte, s'anomena *master*.

8. Sincronitzar el nostre repositori amb un repositori remot com a origen:

```
git pull https://URLdelRepositori(amb extensió .git)
```

En aquesta sincronització, si hi ha dues versions del mateix fitxer, es respecta la que s'ha modificat des del darrer commit comú.

El problema està quan el mateix fitxer s'ha modificat a tots dos repositoris. Llavors es produeix un **conflicte** que haurem de resoldre manualment.

Un procediment és aquest:

1. Fer el pull:

```
git pull --no-commit https://URLdelRepositori(amb extensió .git)
```

2. Si ens informen d'un conflicte, corregir-lo amb:

```
git mergetool -t vimdiff2
```

3. Un cop corregit, fer el commit amb:

```
git commit -m "missatge de confirmació"
```

9. Crear una branca al repositori local:

```
git checkout -b novaBranca
```

10. Canviar-se de branca:

```
git checkout branca
```

Si després de modificar un arxiu es fa *commit* **només afecta** a la branca actual al moment de fer el *commit*

11. Fusionar la branca actual amb una altra:

```
git merge brancaAfusionarAmbLactual
```

Per resoldre els conflictes, l'ordre *merge* admet també l'opció *–no-commit*. En aquest cas cal procedir de manera anàloga a com ho feiem a l'apartat 8.

12. Explorar un commit anterior:

```
git checkout "missatgeCommitAnterior"
```

Per tornar al commit actual només hem de repetir l'ordre tot escrivint el nom del commit actual.

13. Explorar un fitxer concret d'un commit anterior:

```
git checkout "missatgeCommitAnterior" fitxer
```